



Think Parallel with OpenMP

Without stepping into your C or C++ based application's algorithmic design, you can still add threads to your existing code to achieve performance upgradation on a multicore processor. How, you may ask. This article tells you!

Do you like your existing sequential C or C++ code multi-threaded so you can run the performance-sensitive codes on multicore processors? And do you want all this with minimal alteration to your code?

Hmm... well, have you ever heard of OpenMP? Let's look at how it can help you.

OpenMP (Open Multi-Processing) is an open standard for writing shared memory parallel programming in C, C++ and Fortran, in the form of mainly compiler directives, library routines and environment variables. This open standard is supported by multiple architectures/platforms including UNIX/Linux and Windows NT. The Architecture Review Board (<http://www.openmp.org>) that owns and maintains the OpenMP brand is a non-profit corporation that has AMD, Cray, Fujitsu, HP, IBM, Intel, NEC, The Portland Group Inc., SGI, Sun Microsystems and Microsoft as its members.

I assume readers are familiar with parallel

programming. As a developer working in C language, my obvious choice of language to show you OpenMP program examples is C, but with little (or rather, no) tweaking, any C++ programmer can also use them. GCC 4.3.2 or higher is required (a whole list of supported compilers can be found at <http://openmp.org/wp/openmp-compilers>). You can use any compiler that supports OpenMP; however, I will use GCC for all my examples in the article.

The OpenMP programming model

OpenMP follows the fork-join parallel execution model. An OpenMP program starts its execution as a single thread (called initial thread) and assumes the execution sequentially. This sequential program gradually evolves as a parallel program, as a 'team of threads' is created when the master (or initial) thread comes across a *parallel* construct in the code to share the workload. Each task is assigned and tied to a different